

Propuesta de Integración

Ejecución Dinámica para Desafíos Prácticos

De: Grupo 5 (Tema 05 - Desafíos Prácticos)
Para: Grupo 8 (Tema 06 - Sandbox / Runtime)

Septiembre 2026

1. Visión General: El Desafío de la Flexibilidad

Como responsables del motor de evaluación (Grupo 5), nos enfrentamos a la necesidad de evaluar distintos tipos de desafíos prácticos: algoritmos, completado de código, validaciones de arquitectura (ArchUnit) y refactorización (PMD/Checkstyle). Cada uno de estos ejercicios requiere comandos de compilación y ejecución radicalmente distintos. Si el **Grupo 8** asume la responsabilidad de saber *cómo* ejecutar cada tipo de desafío a través de una "spec fija" o perfiles hardcodedos, esto generará un acoplamiento fuerte. Cada vez que la cátedra pida una nueva herramienta de validación, el Sandbox tendrá que modificar su código y redespargar.

2. Opciones de Ejecución Dinámica

Nuestra propuesta busca **cero fricción y cero mantenimiento para el Grupo 8**, manteniendo intacta su excelente arquitectura de aislamiento (API → Worker → Ejecutor por stdin). Existen tres alternativas para lograr esto sin que el Grupo 8 deba conocer la lógica académica de cada ejercicio.

Opción 1: Catálogo de Perfiles Dinámico (Almacenado por G8)

El Grupo 8 mantiene en su base de datos un catálogo de perfiles (scripts de ejecución autorizados). El Grupo 5 registra o actualiza estos perfiles a través de nuevos endpoints expuestos por el Grupo 8. En tiempo de ejecución, el Grupo 5 solo envía el código del alumno y el profileId. El Worker del Grupo 8 busca el script asociado, lo inyecta virtualmente y lo ejecuta.

- **Ventaja para G8:** Obtienen el control y la capacidad de auditar los scripts que corren en sus contenedores, ya que residen en su propia base de datos.
- **Ventaja para G5:** Retenemos la flexibilidad de modificar lógicas de evaluación sin depender de despliegues en el Sandbox, invocando simplemente la API de perfiles.

Opción 2: Array de Comandos en el Contrato

En lugar de archivos, el JSON incluiría un atributo "commands": [...]. El Worker del Grupo 8 iteraría sobre este array y ejecutaría los comandos secuencialmente en el contenedor.

Esto también permite auditar los comandos en tiempo real, aunque dificulta la ejecución de lógicas condicionales o scripts más elaborados.

Opción 3: El Script Universal (run.sh) Inyectado Directamente

El **Grupo 5** inyecta el archivo run.sh como un archivo más dentro del array de files del payload enviado por API. Nosotros generamos el script dinámicamente en nuestro backend y el Sandbox simplemente desempaqueta y ejecuta lo que llega.

3. Integración de Seguridad y el ENTRYPOINT (El Nonce)

El Grupo 8 ha implementado un mecanismo de seguridad vital: el uso de un "nonce" para garantizar que el alumno no pueda falsificar el reporte de resultados imprimiendo marcas falsas en la salida estándar (stdout).

Es importante destacar que, **independientemente de cuál de las tres opciones elijamos, el ENTRYPOINT principal del contenedor siempre pertenecerá al Grupo 8.**

La arquitectura funcionará en dos capas para garantizar la seguridad:

- **Capa 1 (Defensa de G8):** El ENTRYPOINT de la imagen Docker es un script propio del Grupo 8 (ej. /sandbox-entryptpoint.sh). Este script se encarga de:
 1. Leer el nonce secreto provisto por el Ejecutor a través de stdin.
 2. Extraer el TAR de archivos en memoria.
 3. Imprimir la marca de inicio segura (ej. ---SANDBOX-[nonce]-INICIO---).
 4. **Llamar a la capa de evaluación** (ya sea ejecutando el perfil de la Opción 1, el array de la Opción 2, o el ./run.sh de la Opción 3).
 5. Imprimir la marca de fin segura (ej. ---SANDBOX-[nonce]-FIN---).
- **Capa 2 (Evaluación de G5):** Los scripts o comandos que proveemos (Opción 1, 2 o 3) se dedican exclusivamente a compilar, correr tests y devolver un JSON con la nota. **Nuestra capa desconoce por completo la existencia del nonce.**

4. Beneficios y Garantías para el Grupo 8

Beneficio	Impacto Arquitectónico
Control y Auditoría	Ustedes siguen dictando el Entrypoint (para validar el Nonce) y controlan los flags de aislamiento (--network none). Si elegimos la Opción 1, también tienen visibilidad total de los scripts antes de correrlos.
Mantenimiento Cero (IaaS)	Toda la lógica académica y las recetas de herramientas se mantienen en el dominio del Grupo 5, sin requerir redespliegues del Sandbox.

5. Contratos de Integración Propuestos

API de Gestión (Si acordamos la Opción 1)

```
PUT /api/v1/sandbox/profiles/java21-junit
{
  "name": "Java 21 - JUnit Runner",
  "image": "sandbox-java:21-tools",
  "entrypointScript": "#!/bin/sh\njavac -d out $(find src -name '*.java')\njava -jar\n/libs/junit-platform-console-standalone.jar --class-path out --scan-class-path\n--details=testfeed"
}
```

Payload de Ejecución (Soportando las diferentes opciones)

```
{
  "executionId": "req-99ab-45bc",
  "challengeId": "ch-alg-902",
  "limits": {
    "timeoutMs": 5000
  }
}
```

```

    },
    // Para la Opción 1 (Catálogo):
    "profileId": "java21-junit",

    // O bien, para la Opción 2 (Comandos explícitos):
    // "commands": ["javac -d out ...", "java -jar ..."],

    // O bien, para la Opción 3 (run.sh inyectado, requiere pasar el image):
    // "image": "sandbox-java:21-tools",

    "files": [
        {
            "path": "src/main/java/com/tup/Solution.java",
            "content": "public class Solution {\n // Código del alumno...\n}"
        },
        {
            "path": "src/test/java/com/tup/SolutionTest.java",
            "content": "import org.junit.jupiter.api.Test;\n// Test oculto inyectado por G5..."
        },
        // Para Opción 3, aquí iría también el objeto del run.sh
    ]
}

```

6. Imágenes de Contenedor y Dependencias

El **Grupo 8** construirá las imágenes base livianas (ej. Alpine) preinstalando en `/libs/` las dependencias necesarias que solicite el Grupo 5 (JUnit, ArchUnit, PMD).

```
FROM eclipse-temurin:21-jdk-alpine
```

```
RUN addgroup -S sandboxgroup && adduser -S sandboxuser -G sandboxgroup
```

```
WORKDIR /app
```

```
RUN mkdir -p /libs
```

```
# Dependencias solicitadas por Grupo 5
```

```
COPY tools/junit-platform-console-standalone.jar /libs/
```

```
COPY tools/archunit.jar /libs/
```

```
COPY tools/pmd-bin/ /libs/pmd/
```

```
# Script de defensa de G8 (manejo de Nonce)  
COPY sandbox-entripoint.sh /sandbox-entripoint.sh
```

```
RUN chown -R sandboxuser:sandboxgroup /app /libs /sandbox-entripoint.sh  
USER sandboxuser
```

```
# Entrypoint exclusivo de G8 que luego invoca los comandos de G5  
ENTRYPOINT ["/bin/sh", "/sandbox-entripoint.sh"]
```

7. Conclusión

Esta integración desacopla perfectamente nuestras responsabilidades. El **Grupo 8** asegura el aislamiento, el manejo seguro del nonce para evitar falsificaciones y la gestión de recursos; mientras que el **Grupo 5** orquesta la evaluación técnica y dicta de manera dinámica (ya sea por perfiles o scripts) cómo se evalúa el código del alumno.